

AerobicWithMe

תוכן עניינים

שם המגיש : תומר פולסקי

2	כללי.....
3	ייעוד המערכת.....
3	דרישות המערכת.....
6	מרכיבי המערכת
7	משתמשי האפליקציה
8	Use Case דיאגרמת
11	מדריך למשתמש-SmartRun.....
12	תיאור האפליקציה

כללי

אפליקציה זו מתאימה למתאמנים מתחילים ומתקדמים שרוצים להשתפר בזמני הריצה שלהם ולשתף ו\או לבנות מסלולי ריצה עם מתאמנים אחרים .

מדובר באפליקציה מבוססת מיקום שבה המשתמש יכול לבנות מסלול מוגדר מראש בעזרת גישה ל-GOOGLE MAPS, לעלות את המסלול לשרת (בפרוייקט זה נעזרתי ב-REALM שמבוסס על MONGO DB). בנוסף לכל משתמש שבנה מסלול ישנה את האפשרות לערוך את המסלול בין אם זה שינוי הכתובות של נקודות הציון על המפה או שינוי המסלול לחלוטין או לחילופין להסיר את המסלול מהשרת לגמרי (כולל מחיקת הרשומות שנשמרו עבור אותו המסלול). לכל משתמש יש גישה לעריכה רק של המסלול שאותו המשתמש בנה וצפייה במסלולים של משתמשים אחרים . יש לציין שכל משתמש יכול לעבור על מסלולים של שאר המשתנים ויכול לעבור על המסלולים האלה ולעלות את זמני הריצה לאפליקציה ועל ידי כך להשוות זמנים עם שאר משתמשי האפליקציה. בנוסף את השוואת הזמנים ומשתמשי האפליקציה שרצו את המסלול ניתן לראות בדף המסלול .

ייעוד המערכת

האפליקציה נועדה למשתמשי אנדוראיד שרוצים לבנות מסלולי ריצה ולבצע השוואה מקוונת עם שאר המשתמשים. האפליקציה תאפשר ריכוזיות המידע, גישה נוחה ומהירה אל רשומות המסלולים ששאר המשתמשים יצרו ואל רשומות של זמני הריצה, גיבוי וייצוא של הנתונים בעזרת MONGODB, בדגש על פשטות התפעול, יציבות המערכת וזמינותה הגבוהה.

דרישות המערכת

- המערכת תספק למשתמשי האפליקציה
 - הוספה/ הסרה/ עידכון של רשומה שמתארת מסלול ריצה .
 - הוספה/ הסרה/ עידכון של רשומה שמתארת את זמני הריצה של משתמשי האפליקציה.
 - צפייה ברשומות של משתמשי האפליקציה לפי מסלול הריצה שהם ביצעו .
 - שמירת הרשומות באופן מקוון בעזרת MONGO DB לשרת ייעודי .
 - גישה מבוקרת אל האפליקציה תתבצע באמצעות שם משתמש וסיסמא, ותאפשר לכל משתמש באפליקציה לערוך/לשנות/ אך ורק את הרשומות שלו וצפייה ברשומות של שאר המשתמשים ללא אפשרות לערוך/לשנות אותם.
 - צפייה בזמן אמת במיקום של המשתמש בעזרת GOOGLE MAPS, כך שהמשתמש ידע היכן והאם הוא נמצא במסלול הנכון. והצגת המרחק של המסלול הנבחר .
- המערכת תספק ממשק נוח ופשוט למשתמשי האפליקציה .
- גיבויים :

✓ גיבוי רשומות מסלולי הריצה וואו זמני הריצה של המשתמשים יתרחשו באופן מיידי ויעלו לשרת בזמן של פחות מ0.5 msec .

• זמני תגובה-פעולות של מעבר מעמוד של המפה של GOOGLE MAPS לרשומות יארכו לכל היותר שנייה אחת .

• מיון של רשומות של זמני הריצה לפי זמן ריצה, תאריך העלאה של הרשומה לשרת, לפי שם המשתמש (לפי סדר אלפבתי)

• אפשרויות השרת

✓ בעזרת השימוש בREALM שמבוסס על

MONGODB, ישנה אפשרות לשרת של

האפליקציה צפייה/עריכה/מחיקה/הוספה של כל הרשומות של כל המשתמשים, בין אם זה רשומות של המסלולים וואו זמני הריצה שלהם .

✓ אפשרות למחיקת/הסרת/עריכה/חסימה של משתמשי מהאפליקציה.

✓ קביעה של אופן ההיתכרות של המשתמש לאפליקציה, אופני החיבור יכולים להיות :

❖ כתובת מייל וסיסמא.

❖ משתמש פייסבוק וסיסמא.

❖ אימות של GOOGLE .

❖ אימות של APPLE ID .

❖ מפתחות API ייעודיים .

❖ אימות של JWK URI .

• האפליקציה תהיה גמישה לשינויים עתידיים והרחבת פונקציונאליות.

• האפליקציה לא תספק את היכולות הבאות בשלב הנוכחי :

○ מיון זמני ריצה של המשתמשים לפי מסלול.

○ מיון סיווג המסלולים לפי פרמטרים של אורך

המסלול, סוג הפעילות האירובית (למשל הליכה).

○ עריכת פרטי המשתמש עצמו.

- שמירת מידע היסטורי על מסלולים שהוסרו מהאפליקציה והאפשרות לשחזר אותם .

מרכיבי המערכת

בסיס הנתונים

בסיס הנתונים ימוקם בשרת ייעודי של MONGODB ATLAS .

סנכרון נתונים

האפליקציה נעזרת ב REALM שמבוססת על MONGO DB ATLAS, מה שמאפשר את היתרונות הבאים:

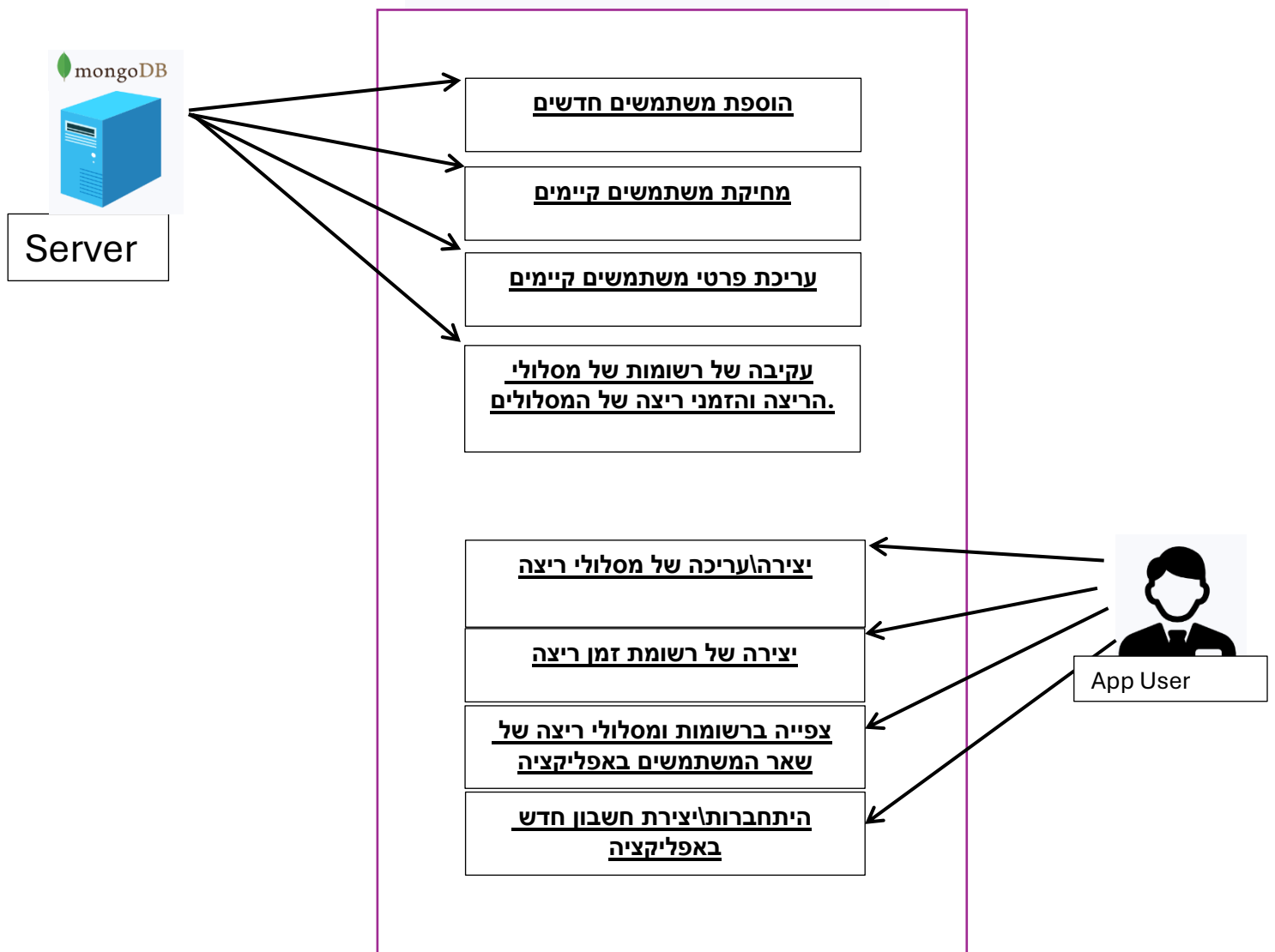
- ✓ הנתונים מתעדכנים בזמן אמת גם כאשר אין חיבור לאינטרנט (offline-first), כך שהאפליקציה ממשיכה לעבוד והסנכרון מתבצע כשחוזר החיבור.
- ✓ איפשר סנכרון אוטומטי בין מסד הנתונים הלוקאלי לבין השרת.

משתמשי האפליקציה

משתמשי האפליקציה יכולים להירשם ולהיתחבר בעזרת אימייל סיסמא שנשמרת בשרת של MONGODB ATLAS. למשתמשי האפליקציה האפשרות לבצע את הפעולות הבאות :

- ✓ יצירת מסלולי ריצה והעלתם לשרת.
- ✓ יצירת רשומה של זמן הריצה לפי מסלול(כלומר להפעיל טיימר ולרוץ את המסלול).
- ✓ עריכת שמות הנקודות של מסלולי הריצה.
- ✓ מיון של רשומות של זמני הריצה של המשתמשים לפי :
- תאריך העלאה של הרשומה לענן.
- זמני הריצה מהנמוך ביותר עד לגבוהה ביותר.
- שם המשתמש לפי סדר אלפאבתי.

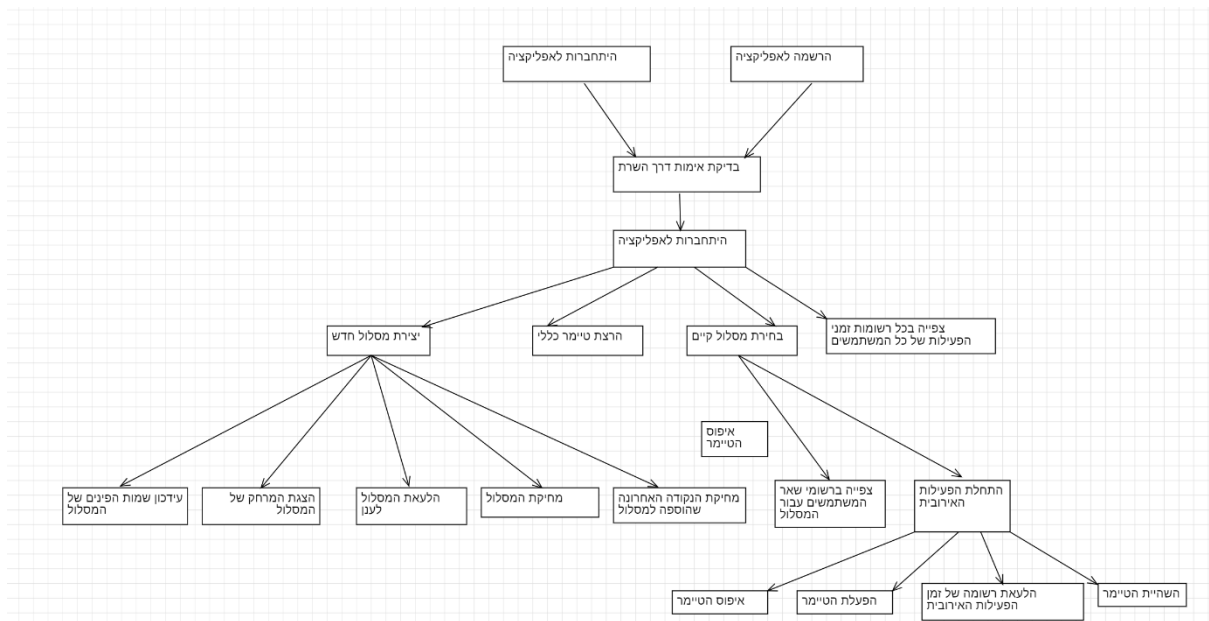
דיאגרמת Use Case ניהול משתמשים באפליקציה



1. משתמש מתחבר/נרשם לאפליקציה.
2. המשתמש יוצר מסלול חדש .
3. משתמש בוחר מסלול קיים .

4. המשתמש יכול לצפות ברשומות זמני
הפעילות של שאר המשתמשים, לצפות בזמני
הפעילות לפי משך הפעילות.

דיאגרמת Activity



מדריך למשתמש-AerobicWithMe

שם המגיש : תומר פולסקי

12	תיאור האפליקציה
14	דגשים והנחות לגבי האפליקציה
15	היתכבות והרשמה לאפליקציה
16	מסך ראשי של האפליקציה
18	מסך הטיימר
19	מסך העלאת הרשומה לענן
20	מסך התצוגה לאחר בחירת כפתור היצירת מסלול חדש
22	מסך הוספת שם למסלול והעלאת המסלול לשרת
23	מסך חישוב המרחק של המסלול
24	עריכת כתובת של נקודה
25	בחירת מסלול מהרשימה
26	מסך הרשומות של זמני הפעילות האירובית לפי בחירת המסלול
27	מיון רשומות של זמני הפעילות האירובית

תיאור האפליקציה

האפליקציה מיועדת (לפחות כרגע) למשתמשי ANDROID בלבד. מדובר על אפליקציה שמאפשרת בעזרת חיישן המיקום של הפלאפון תוך כדי שימוש GOOGLE MAPS בכדי ליצור מסלולי ריצה הליכה וכל פעילות אירובית אחרת. האפליקציה מאפשרת לבצע השוואות של זמני הפעילות האירובית של שאר המשתמשים עבור אותה המסלול. האפליקציה מאפשרת למצוא מיקום של המשתמש בזמן שהוא רץ את המסלול וכמו כן להפעיל טיימר ולעלות רשומה של זמני הריצה של המסלול והוספת תגובות לרשומה. לאחר סיום עריכת הרשומה של זמן הריצה ניתן לעלות את הרשומה לשרת של MONGO DB ATLAS. ולאחר מכן עבור כל מסלול ניתן לראות את הרשומות של שאר המשתמשים ולהשוות זמני ריצה.

מסמכי האפיון הניתוח והעיצוב יתארו את המערכת עבור משתמש חדש שרק נרשם לאפליקציה.

- 1) האפליקציה נבנתה עבור משתמשי אנדרואיד עבור חובבי ריצה הליכה ופעילות אירובית בחוץ.
- 2) משתמשי האפליקציה-בעלי פלאפון מסוג אנדרואיד שמעוניינים לבצע פעילות אירובית במסלולים מוגדרים מראש והשוואת זמני הפעילות עם שאר משתמשי האפליקציה.
- 3) יכולות האפליקציה

- רישום משתמשים חדשים באפליקציה.
- היתחברות של משתמשים קיימים לאפליקציה.
- יצירת מסלולים חדשים בעזרת GOOGLE MAPS.
- עריכה של מסלולים קיימים ושינוי שמות הנקודות של המסלול.
- טיימר מובנה באפליקציה שמאפשר לראות בזמן אמת את זמן הריצה של המשתמש שמחובר כעת לאפליקציה.
- צפייה בזמני הריצה לפי מסלולים של שאר משתמשי האפליקציה.
- מחיקת מסלולים קיימים ומחיקת הרשומות של אותה המסלול.
- מיון של רשומת זמני הריצה של המשתמשים לפי :

- ✓ תאריך העלאה של הרשומה לענן.
- ✓ זמני הריצה מהנמוך עד לגבוהה.
- ✓ שם המשתמש לפי סדר אלפאביתי.

דגשים והנחות לגבי האפליקציה

- המשתמש חייב להירשם לאפליקציה .
- שם המשתמש אינו ניתן לעריכה.
- הסיסמא של המשתמש (נכון לגירסא זו) אינה ניתנת לעריכה.
- האפליקציה נבנתה עבור משתמשי אנדרואיד בלבד.
- לשם העלת הרשומות של המסלולים לשרת יש לוודא שישנו חיבור WIFI תקין .
- בשביל לבנות מסלול חדש חייב שהמיקום של הפלאפון יהיה מאופשר על ידי המשתמש .
- במידה והמשתמש מנסה לעלות רשומה לשרת ולא קיים חיבור WIFI, האפליקציה תעלה באופן אוטמטי את הרשומה ברגע שחיבור הWIFI יחודש.
- במידה והמשתמש מנסה לעלות מסלול לשרת ולא קיים חיבור WIFI, האפליקציה תעלה באופן אוטמטי את המסלול ברגע שחיבור הWIFI יחודש.
- במידה והמשתמש ירצה ליצור מסלול חדש, חייב שיהיה לפחות שני נקודות במסלול .
- השם של המסלול והשם משתמש של רשומת זמן הפעילות חייבים להיות לא ריקים .
- רק משתמש שיוצר מסלול יכול לערוך אותו אחרת תקפוץ הודעת אזהרה .
- במידה והמשתמש ירצה למחוק מסלול שהוא בנה הוא ימחק גם את כל הרשומות של שאר המשתמשים שרצו את המסלול הזה (מופיע הודעת אזהרה לפני מחיקת מסלול).
- זמני הריצה מופיעים בפורמט HH: MM: SS, ואילו תאריך העלאה יהיה בפורמט dd/MM/yyyy .

היתחברות\הרשמה לאפליקציה

20:10 100%

Login

tomer1

.....

Create account

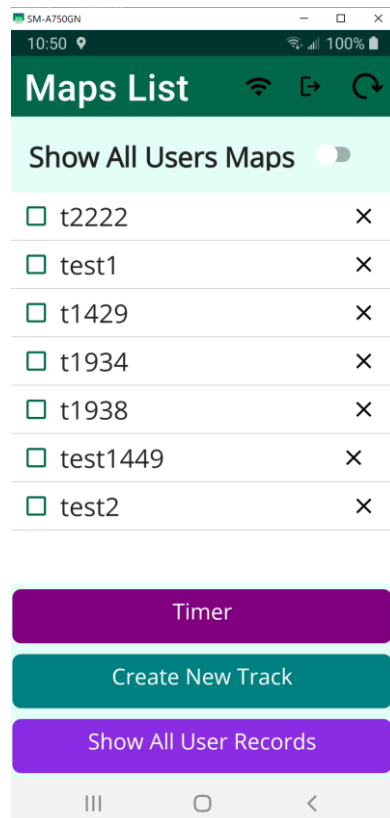
Already have an account?
Log in

Please log in or register with a
Device Sync user account. This

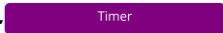
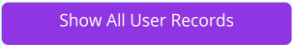
- ניתן להירשם לאפליקציה בעזרת הזנת האימייל או שם משתמש והסיסמא ואז לחיצה על כפתור ה- Create account.
- מסך זה מאפשר להיתחבר לאפליקציה באמצעות שם משתמש וסיסמא על ידי לחיצה על כפתור ה- Already have an Account.

מסך ראשי של האפליקציה

לאחר שהמשתמש היתחבר לאפליקציה יופיע לו המסך שבו מופיעים כל המסלולים ששאר המשתמשים העלו לענן.



מסך זה כולל 3 כפתורים וגילימים והרשימה של המסלולים הקמיים בשרת.

-  -כפתור המעביר את המשתמש למסך של טיימר .
-  - מסך שמעביר את המשתמש לעמוד של GoogleMaps ומאפשר למשתמש ליצור מסלול חדש .
-  -מסך שמראה את כל רשומות זמני הריצה של כל המשתמשים .

3 כפתורי תמונות

-כפתור זה בעצם מרענן את הרשימה של המסלולים (במידה ומשתמש אחר הוסיף מסלול חדש) אז ניתן לראות את המסלול.



-כפתור זה מאפשר להיתנתק מהאפליקציה ולחזור למסך של ההיתחברות .



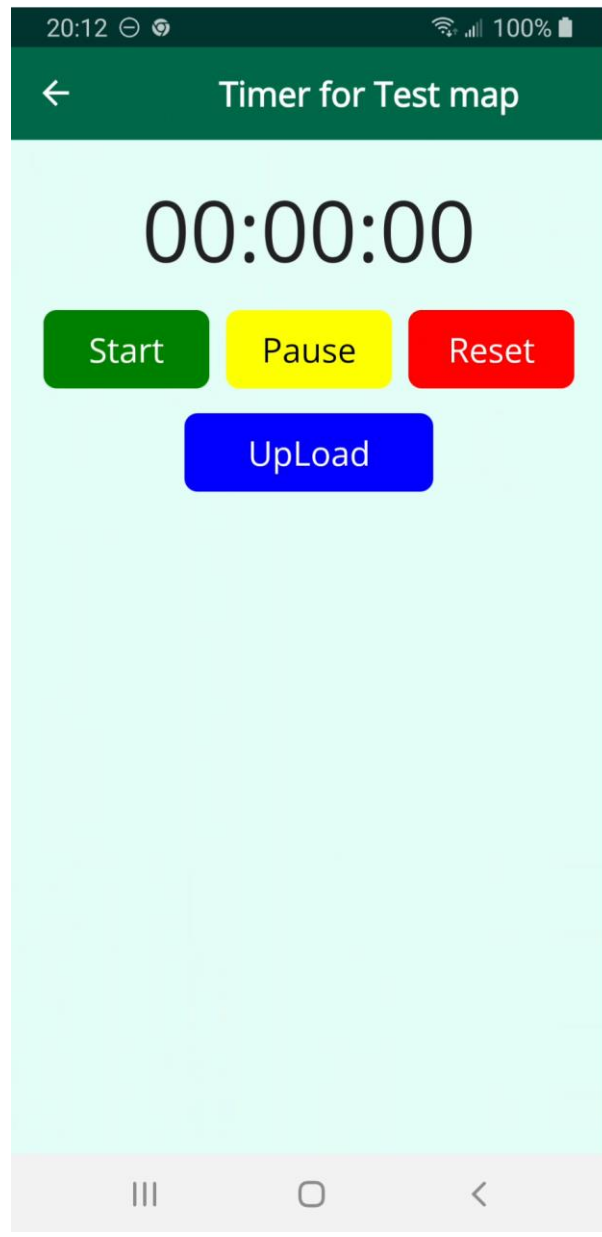
-כפתור שמאפשר לכבות את החיבור של הWIFI באפליקציה .



-מפסק שמאפשר לראות את כל המסלולים של המשתמש בלבד או של כל המשתמשים שהעלו מסלולים לאפליקציה .



מסך הטיימר



- מסך זה מאפשר להפעיל טיימר ללא תלות בבחירת המסלול .
- מסך זה כולל כפתורים של התחלת הספירה של הטיימר, השהייה של הטיימר, ואיפוס הטיימר .
- UPLOAD-כפתור שמאפשר לעלות לענן את הזמן של המשתמש לענן .

מסך העלאת הרשומה לענן

SM-A750GN

12:14 57%

← Map: Test1316

User Name: tomer

Comments:
Enter your comments here

MapName: Test1316

Record Time: 00:00:00

Upload Time: 12:14:30

Upload Date: 30-01-2025

OK Cancel

- מסך זה מאפשר לעלות את את הרשומה של המשתמש לענן.
- מסך זה כולל את השם משתמש שהמשתמש יכול לקבוע, הערות שהמשתמש יכול להוסיף לרשומה, שם המפה, זמן הפעילות האירובית, השעה והתאריך של סיום הפעילות.
- לחיצה על כפתור ה-OK יוסיף את הרשומה לענן.

מסך התצוגה לאחר בחירת כפתור היצירת מסלול חדש

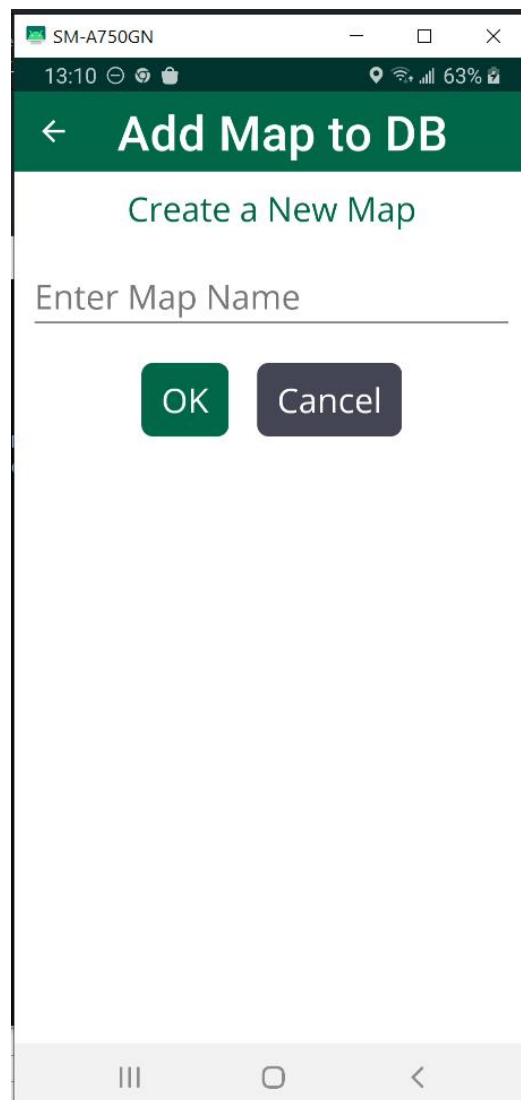


לחיצה על גבי המפה תוסיף נקודה על המפה שנראית כך: . על גבי המסך של יצירת מסלול חדש בעזרת GOOGLE MAPS.

-  -כפתור שמעביר את המשתמש למסך שבו ניתן לערוך את מספר הנקודה ולשנות את הכתובת, של הנקודה. ברירת המחדל שהכתובות של הנקודות הם מספרים שמתחילים מ1.
-  - כפתור שמאפשר למחוק את המפה שהמשתמש יצר לפני שהוא העלה אותה לענן.
-  - כפתור שמעביר את המשתמש למסך שמציג את המרחק מנקודה לנקודה, והמרחק הכללי של המסלול שהמשתמש בנה עד עכשיו.
-  -כפתור שמעביר את המשתמש למסך של העלאת המסלול.

- -כפתור שמבצע זום למיקום של המשתמש ברגע זה.
- -כפתור שמאפשר למחוק את הנקודה האחרונה שהוספה למסלול.

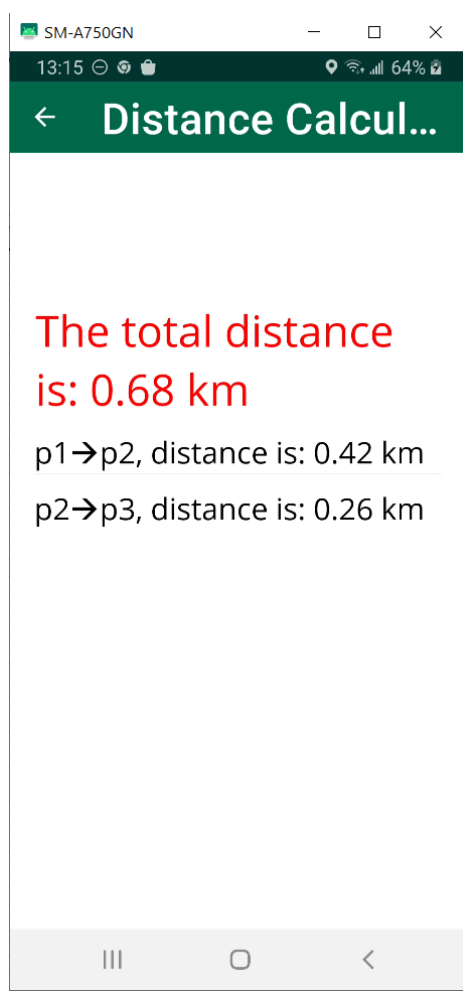
מסך הוספת שם למסלול והעלאת המסלול לשרת



The screenshot shows a mobile application window titled "Add Map to DB". At the top, there is a green header bar with a back arrow and the title. Below the header, the text "Create a New Map" is displayed. Underneath, there is a text input field labeled "Enter Map Name". At the bottom of the input area, there are two buttons: a green "OK" button and a grey "Cancel" button. The status bar at the top of the phone shows the time as 13:10, battery level at 63%, and various connectivity icons. The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps buttons.

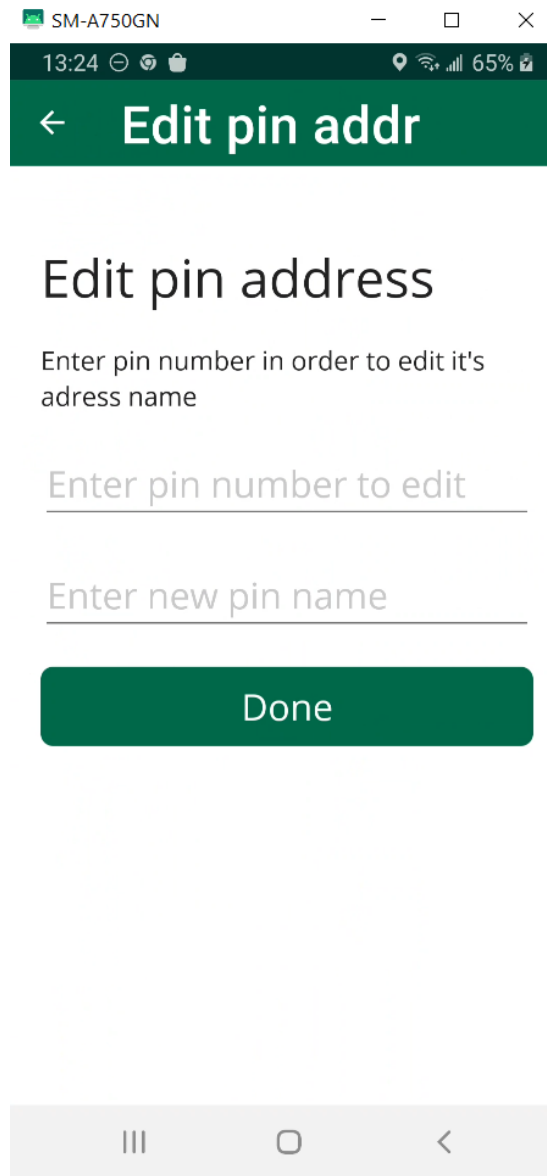
במסך זה ניתן לבחור את שם המסלול ולאחר לחיצה על כפתור ה-OK לעלות את המסלול לענן של ATLAS MONGODB DB.

מסך חישוב המרחק של המסלול



מסך זה מאפשר לראות את המרחק מנקודה לנקודה של המסלול ואת אורך המסלול הכללי. כמובן שהנקודה הראשונה של המסלול מסומנת P1 השנייה P2 וכך הלאה.

עריכת כתובת של נקודה



SM-A750GN

13:24 65%

← Edit pin addr

Edit pin address

Enter pin number in order to edit it's adress name

Enter pin number to edit

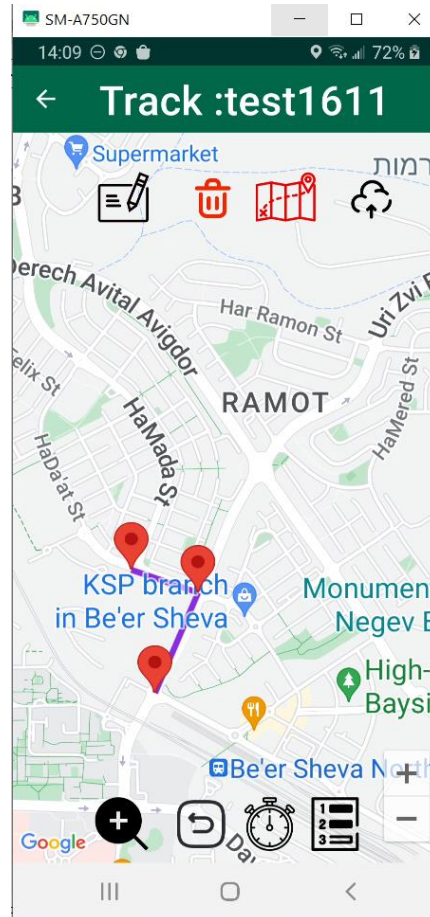
Enter new pin name

Done

מסך זה בעצם מאפשר לערוך את הכתובת של נקודה מסויימת, על ידי לחיצת מספר הנקודה ואז הכנסת שם כתובת חדש.

בחירת מסלול מהרשימה

במידה והמשתמש החליט לבחור מסלול שהוא בנה יופיע המסך הבא :

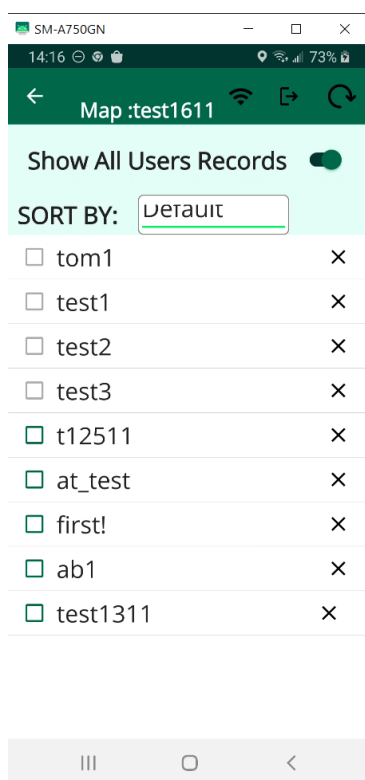


כפתורי המסך יהיו זהים לכפתורים של המסך לאחר לחיצה על הכפתור שיוצר מסלול חדש, פרט ל2 הכפתורים הבאים :

2. -כפתור הטיימר שמעביר את המשתמש למסך הטיימר .

3. -כפתור שמאפשר למשתמש לצפות ברשמות זמני הפעילות האירובית בהתאם למסלול הנבחר .

מסך הרשומות של זמני הפעילות האירובית לפי בחירת המסלול



מסך זה בעצם מאפשר לראות זמני הרשמות של הפעילות האירובית של שאר המשתמשים בהתאם למסלול שאותו בוחרים.

כפתור זה מאפשר להראות את כל הרשמות של המתמשים שביצעו את הפעילות האירובית של המסלול, או אם הכפתור כבוי להראות רק את הרשומות של המשתמש שמחובר כעת לאפליקציה. ניתן לראות את הרשמות של המשתמש שמחובר כעת לאפליקציה בעזרת הריבוע הירוק בצד שמאל, וניראת כך: ☐, רשומה של משתמש אחר צבועה בצבע אפור ונראית כך: ☐.

× - לחיצה על כפתור הX תמחוק את הרשומה מהשרת(ניתן למחוק אך ורק רשומות של המשתמש שמחובר כרגע לאפליקציה)

מיון רשומות של זמני הפעילות האירובית

לחיצה על האפשרות של SORT BY . מאפשרת למיין את הרשומות
. לאחר הלחיצה יופיע לי החלון הבא :

Choose an option

Profile Name

Record Time

Upload Date

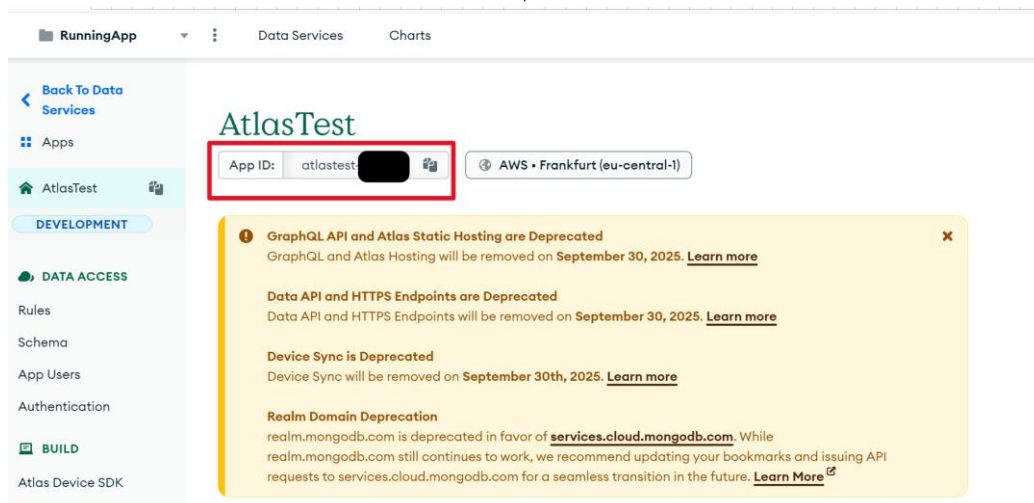
Default

Cancel

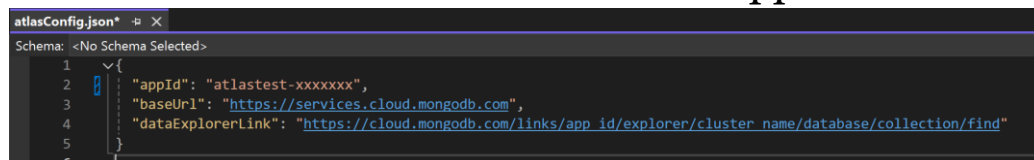
ניתן למיין את הרשומות לפי שם הפרופיל,משך הפעילות האירובית
,תאריך העלאת הרשומה לענן , או ברירת מחדל.

צד השרת

האפליקציה מבוססת על Mongo DB atlas, שהוא ענן שמיועד להעלעת אובייקטים לענן של MONGO שמיועד בעיקר לאפליקציות ANDORID . בשביל לראות את המשתמשים והאובייקטים שעלו לענן יש צורך להירשם ללינק הבא :
<https://services.cloud.mongodb.com/groups/66ae39a02e42444be6f2b535/apps>
לאחר ההרשמה יופיע החלון הבא :



את ה AppId שהיתקבל יש להעתיק לקובץ ה-atlasConfig.json תחת שדה ה AppId



לאחר השלמת הצעדים למעלה, ניתן יהיה להיתחבר לשרת של MONGO DB בעזרת דפדפן האינטרנט ולעקוב אחר האובייקטים שעלו לשרת .

משתמשי האפליקציה

את משתמשי האפליקציה ניתן לראות על ידי לחיצה על app users

Users									
Users		User Settings		Authentication Providers					
Confirmed		Pending		Providers		Filter by enabled		Find a user by ID...	
								Apply	
Name	ID	User Type	Providers	Created Date	Last Login Date	User Status	Enabled	Actions	
tomor1	69d5b72c5e9dbae278c8b92afdf	normal	Email/Password	08/02/2024 13:01:32 UTC	01/30/2025 11:22:37 UTC	Confirmed	On	VIEW ACTIVITY	...
tomor00	672f45c4346222907f61a2b05	normal	Email/Password	10/28/2024 08:05:24 UTC	01/27/2025 14:11:07 UTC	Confirmed	On	VIEW ACTIVITY	...
tomor2	675b2575b6fc13f6e986c47211	normal	Email/Password	12/12/2024 16:57:25 UTC	12/12/2024 16:57:25 UTC	Confirmed	On	VIEW ACTIVITY	...
tomor5	675b26cbbcf0d422ccfbafcf2	normal	Email/Password	12/12/2024 17:01:31 UTC	12/29/2024 16:43:46 UTC	Confirmed	On	VIEW ACTIVITY	...
tomor11	67792e79a2f12a9d6ea2c452	normal	Email/Password	01/04/2025 11:26:49 UTC	01/23/2025 12:55:56 UTC	Confirmed	On	VIEW ACTIVITY	...
tomor01	677992273a9c38366713e42e	normal	Email/Password	01/04/2025 15:47:39 UTC	01/04/2025 15:47:39 UTC	Confirmed	On	VIEW ACTIVITY	...
LOAD MORE									

הרשמה לאפליקציה

את אופן הרישום לאפליקציה ניתן לבחור על ידי לחיצה על ידי לחיצה על Authentication Providers, את האפשרויות שבעזרת ניתן להירשם לאפליקציה ניתן לראות בתמונה למטה .

Authentication

Users

User Settings

Authentication Providers

App Services provides several authentication providers that you can integrate into a client application to allow users to log in to your app. [Learn more about how to configure authentication for any of our authentication providers.](#)

Provider	Enabled	Edit
Allow users to log in anonymously	On	<button>EDIT</button>
Email/Password	On	<button>EDIT</button>
Facebook	Off	<button>EDIT</button>
Google	Off	<button>EDIT</button>
Apple	Off	<button>EDIT</button>
API Keys	Off	<button>EDIT</button>
Custom JWT Authentication	Off	<button>EDIT</button>
Custom Function Authentication	Off	<button>EDIT</button>

סינכרון האפליקציה

לפני הפעלת האפליקציה יש לוודא שישנו סינכרון של המכשירים לענן של MONGO DB . על ידי לחיצה על לשונית Device Sync .

The screenshot shows the 'Device Sync' configuration page in the MongoDB Atlas interface. The left sidebar contains navigation links for 'Back To Data Services', 'Apps', 'AtlasTest', 'DEVELOPMENT', 'DATA ACCESS', 'BUILD', 'MANAGE', and 'HELP'. The main content area is titled 'Device Sync' and includes a deprecation warning: 'Deprecation of Device Sync. We announced the deprecation of Device Sync in September 2024. This service will be removed on September 30, 2025. Learn More'. Below the warning, there are tabs for 'Dashboard', 'Metrics', 'Logs', 'Sync Models', and 'Configuration'. The 'Configuration' tab is selected, showing 'Select Development Details' where a cluster is chosen. There are also 'Manage Sync' buttons for 'Pause Sync' and 'Terminate Sync'.

אובייקטים שעלו לשרת

על ידי לחיצה על DataServices ואז על DataBase, ניתן לראות את האובייקטים שעלו לענן של MONGO DB .

The screenshot shows the 'RunningAppDataBase' page in the MongoDB Atlas interface. The left sidebar contains navigation links for 'Overview', 'DATABASE', 'Clusters', 'SERVICES', 'SECURITY', and 'Goto'. The main content area is titled 'RunningAppDataBase' and includes a 'Collections' tab. The 'Collections' tab shows a table of collections with the following data:

Collection Name	Documents	Logical Data Size	Avg Document Size	Storage Size	Indexes	Index Size	Avg Index Size
MapPin	31	6.91KB	229B	36KB	1	36KB	36KB
TrackMongol	11	2.35KB	220B	36KB	1	36KB	36KB
UserRecord	27	8.59KB	326B	36KB	1	36KB	36KB

AerobicWithMe

מסמך תכנון ועיצוב
שם המגיש : תומר פולסקי

תוכן עניינים

35	תיאור כללי של המערכת
35	הנחות עבודה בכתיבת הפרוייקט
36	מוסכמות רישום
37	שכבת בסיס הנתונים-Mongo Db Realm
40	האובייקטים שמעלים לשרת
40	1.אובייקט- UserRecord
41	2.האובייקט- MapPin
42	3.האובייקט- TrackMongo1
43	שכבת הלוגיקה
44	מחלקות של אובייקטים-שמעלים לMongoDb
45	מחלקות כלליות
45	MapUtility.cs
46	SetObjectToUpload.cs
47	ממשקים ומחלקות סטטיות
47	I_Command
47	ILogin
47	IMapUtility
48	IRealmService
48	RealmService.cs
50	מחלקות של מודל תצוגה
51	. EditMapPinViewModel.cs
52	. EditUserRecordViewModel.cs
53	. MapsViewModel.cs
54	. UserRecordsViewModel.cs
55	LoginViewModel.cs
56	שכבת ההצגה -קבצי xaml של Views
57	עקרונות מנחים בשכבת ההצגה
58	דפים המשמשים להצגה

59	AddMapToDbPage.xaml.cs
59	.DistancePage.xaml.cs
59	EditPinAddr.xaml.cs
60	MapPage.xaml.cs
61	MapsPage.xaml.cs
61	AddRecordToDb.xaml.cs
62	דפוסי עצוב (Design Patterns)
62	SingletonPattern
63	Command pattern
65	Strategy Pattern
67	Observer Pattern
69	שינויים עתידיים אפשריים
70	דיאגרמות

תיאור כללי של המערכת

המערכת בנויה על סמך מודל 3 השכבות (TIER-3)

- שכבת התצוגה (PL-Presentation Layer)
- שכבת האפליקציה (BL-Buisness Logic Layer)
- שכבת גישה לנתונים (DAL-Data Access Layer)

כל שכבה מתקשרת עם שאר השכבות. ההפרדה לשכבות מאפשרת ליצור פרויקט מסודר שיקל על סידור האפליקציה.

הנחות עבודה בכתיבת הפרויקט

1. האפליקציה פותחה ועובדת אך ורק עבור משתמשי אנדוראיד.
2. משתמשי האפליקציה חייבים לאשר הרשאה למיקומם אחרת האפליקציה לא תעבוד בצורה תקינה.
3. בשביל לעלות רשומות או מסלולים שהמשתמש בנה יש לוודא שישנו חיבור תקין ל-WIFI, אחרת הרשומה תעלה כשהמשתמש יתחבר ל-WIFI בהמשך.
4. הזמנים והתאריכים שנשמרים בענן נשמרים בפורמט הבא: DD/MM/Year Hour: Minutes: Seconds
5. הזמנים והתאריכים נשמרים לפי המדינה שבה משתמש האפליקציה נמצא.
6. אסור ליצור מסלול שלא כולל בו לפחות 2 נקודות.
7. כאשר מעלים רשומה של זמן מסויים עבור מסלול מסויים, שם המשתמש לא יכול להיות ריק.
8. האפליקציה מבוססת על FREE TRAIL של MONGO DB ATLAS, לכן הגודל המקסמלי של כל האובייקטים שניתן לעלות הם עד 36KB.
9. שמות המסלולים הם יחידים, כלומר אסור ששם של מסלול יהיה פעמיים, אחרת תופיע הודעת מערכת שיש להחליף את השם - לשם שלא קיים במערכת.

מוסכמות רישום

1. כל שם מחלקה שכוללת יותר ממילה אחת נכתבת עם אות גדולה בתחילה כל מילה נוספת.
2. שיטות עזר(אלו הן שיטות שמשמשות לחישובים שונים כמו למשל השיטה `getDistance2Points` שמחשבת את המרחק בין 2 נקודות על גבי המפה) במחלקה מתחילה מאות קטנה, במידה ושם השיטה כולל יותר ממילה אחת אז רק המילה הראשונה מתחילה מאות קטנה, שאר המילים מתחילים באות גדולה.
3. שיטות שמשמות בשביל תצוגת ה XAML (כמו למשל `SaveUserRecord` שמשמשת להעלאת רשומה חדשה ל MONGO DB) ושכוללות יותר ממילה אחת יכתבו כל מילה באות גדולה. `Interfaces`.
4. ממשקים של האפליקציה ישמרו בתוך תיקייה שנקראת `Interfaces` ושמות הממשקים יתחיל באות I .
5. מחלקות שהן חלק מ- `Strategy Design Pattern` יתחילו במילה `Sort` .
6. מחלקות שהן חלק מ `Command Design Pattern` יסתיימו במילה `Command` .

שכבת בסיס הנתונים-Mongo Db Realm

שכבה זו אחראית על התקשורת עם המסד נתונים . והיתחברות של המשתמש ל MONGO DB. המערכת עושה שימוש בעזרת מסד נתונים של MONGO DB ATLAS REALM .

שכבה זו מכילה מחלק בשם RealmService שנמצאת בתקיית Services. מחלקה זאת אחראית ליצור את ההיתקשורות עם המסד נתונים בעזרת שימוש בשיטה של GetRealm שנעזרת באובייקטים מהספרייה של Realms. נציין שבשביל לסנכרון את האפליקציה עם הענן של REALMS שפתחנו בMONGO DB ATLAS, יש להעזר בקובץ atalasConfig.json ותחת השדה של appId להדביק את הId שנוצר באתר של MONGO DB ATLAS .

המחלקה מכילה את השיטות הבאות :

GetRealm()-יוצר מופע של Realm בהתאם לסוג האובייקט הפעיל MapPin או UserRecord ומגדיר את הסנכרון המתאים.

RegisterAsync(string email, string password)- רושם משתמש חדש ב MongoDB Atlas עם אימייל וסיסמה.

LoginAsync(string email, string password)- מתחבר לחשבון המשתמש באמצעות אימייל וסיסמה, פותח את Realm ומממין לסנכרון הנתונים.

LogoutAsync()-מתנתק מהמשתמש הנוכחי שמחובר לאפליקציה, סוגר את התהליכון של mainThreadRealm ומוחק את התהליכון הנוכחי .

SetSubscription(Realm realm, SubscriptionType subType)-

- מגדיר את סוג המנוי Mine or All.
 - מוחק את המנוי הקיים ומוסיף חדש בהתאם.
 - ממתין לסנכרון הנתונים אם החיבור אינו מנותק.
- GetCurrentSubscriptionType(Realm realm)- בודק את סוג המנוי הנוכחי (mine או all)ומחזיר את ערכו כ-SubscriptionType.

ומחזירה שאילתת חיפוש (`IQueryable<MapPin>`) ושם מנוי (`string`) עבור אובייקט מטיפוס `MapPin` במסד הנתונים `Realm`, בהתאם לסוג המנוי (`SubscriptionType`).

ומחזירה שאילתת חיפוש (`IQueryable<UserRecord>`) ושם מנוי (`string`) עבור אובייקט מטיפוס `UserRecord` במסד הנתונים `Realm`, בהתאם לסוג המנוי (`SubscriptionType`).

- עקרונות מנחים לגבי שכבת בסיס הנתונים :
בסיס הנתונים מבוסס על `MongoDB Atlas` הוא שירות `Database-as-a-Service (DBaaS)` שמספק `MongoDB` מנוהל בענן. מדובר בפתרון שמאפשר לך להפעיל, לנהל ולתחזק מסדי נתונים של `MongoDB` בענן, ללא צורך בניהול שרתים פיזיים.

מאפיינים עיקריים של `MongoDB Atlas` :

מסד נתונים מבוסס `NoSQL`

- `MongoDB` הוא מסד נתונים מסמכי (`Document-Oriented`) המאחסן נתונים בפורמט `JSON` דינמי (`BSON`).
- מאפשר שמירת נתונים בצורה גמישה ומותאמת ליישומים מודרניים.

תמיכה בפריסה בענן

- ניתן להריץ את `MongoDB Atlas` על `AWS`, `Azure` או `Google Cloud`.
- מאפשר לך לבחור אזורי פריסה (`Regions`) וליצור מסדי נתונים קרובים למשתמשים שלך.

ניהול אוטומטי של מסד הנתונים

- גיבויים אוטומטיים, ניטור ביצועים, איזון עומסים.
- עדכונים אוטומטיים של התוכנה כדי לשמור על אבטחה וביצועים מיטביים.

גמישות וסקלבליות (Scalability)

- ניתן להגדיל או להקטין את המשאבים בהתאם לצרכים העסקיים שלך.
- תמיכה ב Sharding- (פיצול נתונים בין מספר שרתים) ו-Replication שכפול נתונים לגיבוי ואמינות

תמיכה בסנכרון נתונים עם Realm

- מאפשר סנכרון נתונים עם יישומים ניידים או אינטרנט באמצעות MongoDB Realm, פתרון לסנכרון נתונים ביישומים ללא חיבור קבוע לרשת.

פאנל ניהול אינטואיטיבי

- ממשק גרפי (Atlas UI) לניהול מסדי נתונים, שאילתות, הרשאות משתמשים וניטור ביצועים.

האובייקטים שמעלים לשרת

האפליקציה מעלה 3 אובייקטים לשרת. אובייקטים אלו כוללים :

1. UserRecord - רשומה של זמן הפעילות של המשתמש בהתאם לסוג המסלול שהוא בחר .

2. MapPin - אובייקט שמתאר נקודה שהיא חלק ממסלול כך שיהיה ניתן לראות את המסלול על גבי Google Maps

3. TrackMongo - אובייקט ששומר את הפרטים הכלליים של המסלול . כך שלמנהל השרת תהיה גישה יותר נוחה לבדיקת המסלולים

1. אובייקט - UserRecord

Id - מזהה ייחודי של המשתמש במסד הנתונים (MongoDB). בעזרת השיטה `ObjectId.GenerateNewId` נוצר טיפוס חדש בשם `ObjectId`, עבור כל אובייקט שאנו מעלים .

OwnerId - מזהה של המשתמש שהעלה את הרשומה, חובה למילוי .

ProfileName - שם הפרופיל של המשתמש, מטיפוס `STRING`.

MapName - שם המפה המשויכת למשתמש, מטיפוס `STRING` .

TrackTime - משך הזמן של הפעילות האירובית של המשתמש באותו המסלול, מטיפוס `STRING` .

UploadDateTime - התאריך והשעה שבהם הנתונים הועלו למסד הנתונים (MongoDB), מטיפוס `STRING` .

Comment - הערות שהמשתמש הוסיף לרשומה. מטיפוס `STRING`.

IsMine - שדה מחושב, (Read-Only) מחזיר `true` אם המשתמש הנוכחי (ב `RealmService`) הוא הבעלים של הרשומה.

2. האובייקט - MapPin

האובייקט השני שאנו מעלים לענן הוא אובייקט מטיפוס MapPin, אובייקט זה כולל את הנקודות של המסלול שאותו המשתמש מעלה לענן. כאשר יותר מ-2 נקודות על המפה עם אותו השם בעצם יוצרות מסלול שאותו ניתן לעלות לענן.

Id-מזהה ייחודי של הנקודה שהמשתמש הוסיף על המפה, מיוצר אוטומטית. (ObjectId)

OwnerId-מזהה של האובייקט של הנקודה שהוספה על המפה.

Mapname -שם המפה של המסלול. טיפוס מסוג STRING.

Labelpin -תווית (Label) המתארת את הנקודה. טיפוס מסוג STRING.

Address -הכתובת של המיקום שבו נמצאת הנקודה. טיפוס מסוג STRING.

Latitude -קו רוחב (Latitude) של הנקודה שנמצאת על המפה. טיפוס מסוג STRING.

Longitude -קו אורך (Longitude) של הנקודה שנמצאת על המפה. טיפוס מסוג STRING.

3. האובייקט - TrackMongo1

האובייקט השלישי שאנו נירצה לעלות הוא אובייקט שמתאר פרטים כלליים של המסלול , למשל השם שלו, מספר הנקודות של המסלול והאורך של המסלול.

Id-מזהה ייחודי של הנקודה שהמשתמש הוסיף על המפה, מיוצר אוטומטית.(ObjectId)

OwnerId-מזהה של האובייקט של הנקודה שהוספה עלהמפה .

IdOfFirstPin - מזהה ייחודי של הנקודה הראשונה של המסלול שהוספנו .

TrackName-שם המסלול .

DateOfCreation-התאריך שבו נוצר המסלול.

NumberOfPins-מספר הנקודות של המסלול .

DistanceOfTrack-המרחק של המסלול .

שכבת הלוגיקה

שכבת הלוגיקה היא השכבה המרכזית באפליקציה שמכילה את כללי העסק (Business Rules) ואת כל הלוגיקה שמנהלת את האפליקציה שלך. היא המוח של האפליקציה, שמחליט איך הנתונים מעובדים ומנווטים בין שכבות אחרות, כמו שכבת הנתונים (DAL) ושכבת המצגת (UI).

עקרונות כלליים בשכבת הלוגיקה :

- **הפרדת אחריות** – (Separation of Concerns) מחלקים את האפליקציה כך שה UI-מטפל רק בתצוגה, ושכבת הלוגיקה מטפלת בעיבוד הנתונים.
- **קלות תחזוקה** – אפשר לשנות את הלוגיקה העסקית בלי להשפיע על ה UI.
- **שימוש חוזר** – פונקציות בשכבת הלוגיקה יכולות לשמש גם באפליקציות אחרות) למשל, אם בעתיד נבנה גרסה ל ISO ול Windows .
- **בדיקות יחידה** – (Unit Tests) אפשר לבדוק את הלוגיקה בלי להריץ את כל האפליקציה.

מחלקות של אובייקטים- שמעלים לMongoDb

מחלקת MapPin - מחלקה זו מייצגת, נקודה על גבי המפה. וכוללת בתוכה את השדות הבאות :

- Id: ObjectId
- OwnerId: string
- Mapname: string
- Labelpin: string
- Address: string
- Latitude: string
- Longitude: string
- IsMine: bool

מחלקת UserRecord - מחלקה זו כוללת רשומה של משתמש שמבצע משתמש על גבי מסלול שהוא בוחר. המחלקה כוללת את השדות הבאים :

- Id: ObjectId
- OwnerId: string
- ProfileName: string
- MapName: string
- TrackTime: string
- UploadDateTime: string
- Comment: string
- IsMine: bool

מחלקת TrackMongo1 - מחלקה זו כוללת את השם של המסלול, מחלקה זאת נוצרה בשביל להקל על מנהל האפליקציה לראות כמה מסלולים קיימים במאגר.

- Id: ObjectId
- OwnerId: string
- IdOfFirstPin: ObjectId
- TrackName: string
- DateOfCreation : string
- NumberOfPins : int
- DistanceOfTrack : double
- IsMine: bool

מחלקות כלליות

MapUtility.cs

מחלקה זאת היא מחלקת עזר שכוללת את כל החישובים הנכללים בתוך בניית מסלול חדש .

- MapUtility()
- MapUtility(List<Pin> pinsList)
- MapUtility(Map newMyMap)
- MapUtility(List<Pin> NewPinsList,.Map newMyMap)
- void setMap(Maui.GoogleMaps.Map myMap)
- void setPinList(List<Pin> pinsList)
- void addPointsToTrack()
- string getDateTime()
- string getCurrentTime()
- string getCurrentDate()
- void addPointsToTrackOnMap()
- void setPinsList(List<Pin> newpinList)
- void drawLineBetweenAllPins(int strokeColorPolyline)
- Task<bool> enoughPins(int num)
- private void removePolylineBetweenPins(Pin pin1, Pin pin2)
- void deleteLastPoint(List< Pin> pinsList)
- List<string> getPtrNamesAndPolylines()
- private void drawLineBetween2Pins(Pin pin1, Pin pin2, int strokeColorPolyline)
- double calculateTotalDistance()
- private double getDistance2Points(Pin p1,Pin p2)

SetObjectToUpload.cs

מחלקה זו מייצגת מחלקה שבוחרת את טיפוס המחלקה שיעלה לענן .
אחת השיטות שמופיעות במחלקה הם Set שקובעות האם הטיפוס של
המחלקה יהיה מטיפוס UserRecord או MapPin או
TrackMongo . וכוללת את השיטות הבאות :

- void setObjectToUpload()
- void CreateMapPin()
- void CreateUserRecord()
- void CreateTrackMongo()
- Type GetCurrentObjectType()
- object GetCurrentObject()

Track.cs

מחלקה זאת היא מחלקת עזר שבתוכה שומרת את השיטות שבעזרת
נעלה את האובייקטים של MapPin ו TrackMongo1 לשרת . החלקה
כוללת את השיטות הבאות :

- public void Attach(ITrackObserver observer)
- public void Detach(ITrackObserver observer)
- public void Notify()
- public async Task AddTrack()
- public async Task RemoveTrack(MapPin pinOfChosenMap)
- private async Task DeleteSinglePin(MapPin pin)
- private async Task DeleteTrack(MapPin pin)
- private async Task SavePin(Pin newPin, int pinNumber)
- public async Task SaveTrack(ObjectId firstPinIdNumber)

ממשקים ומחלקות סטטיות

כל הממשקים של האפליקציה נמצאים תחת תקיית Interfaces וכוללת בתוכה את הקבצים הבאים :

I Command

מחלקה זאת מייצגת ממשק שממש את הפונקציה של Excute, ממשק זה הינו חלק מדפוס העיצוב של Command.

ILogin

מחלקה זאת מייצגת ממשק אחיד שאחראי על ההיתחברות\הרשמה של משתמש לאפליקציה. וכוללת בתוכה את השיטות הבאות

```
Task OnAppearing();  
Task Login();  
Task SignUp();  
Task<bool> VerifyEmailAndPassword();  
Task GoToMainPage();
```

IMapUtility

מחלקה זאת מייצגת ממשק אחיד שאחראי כל השיטות שקשורות ליצירת מסלול על גבי המפה של Google.Maps. המחלקה כוללת את השיטות הבאות :

```
void addPointsToTrack();  
void addPointsToTrackOnMap();  
void setPinsList(List<Pin> newPinsList);  
void drawLineBetweenAllPins(int strokeColorPolyline);  
void deleteLastPoint(List<Pin> pinsList);  
List<string> getPtrNamesAndPolylines();  
public string getDateTime();  
public string getCurrentTime();  
public string getCurrentDate();  
public Task<bool> enoughPins(int num)
```

IRealmService

מחלקה שכוללת את השיטות הקשורות לחיבור של האפליקציה ל MongoDBAtlas Realm – כאשר Realm הוא מסד נתונים NoSQL המתמקד בפשטות, ביצועים גבוהים, וסנכרון נתונים בזמן אמת. הוא משמש בעיקר לאפליקציות מובייל (iOS, Android, .NET MAUI) וכולל יכולות סנכרון עם הענן. המחלקה כוללת את השיטות הבאות :

```
User CurrentUser { get; }
string DataExplorerLink { get; }
Task Init();
Realm GetMainThreadRealm();
Realm GetRealm();
Task RegisterAsync(string email, string password);
Task LoginAsync(string email, string password);
Task LogoutAsync();
Task SetSubscription(Realm realm, SubscriptionType subType);
SubscriptionType GetCurrentSubscriptionType(Realm realm);
```

RealmService.cs

המחלקה RealmService היא מחלקת שירות סטטית שמנהלת את האינטראקציה עם MongoDB Realm באפליקציה. היא מטפלת באתחול, אימות משתמשים, גישה לנתונים, וניהול מנויים (Subscriptions) וכוללת את השיטות הבאות :

- private static (IQueryable<TrackMongo1> Query, string Name) GetQueryForSubscriptionTrackMongoType(Realm realm, SubscriptionType subType)
- private static (IQueryable<UserRecord> Query, string Name) GetQueryForSubscriptionUserRecordType(Realm realm, SubscriptionType subType)
- private static (IQueryable<MapPin> Query, string Name) GetQueryForSubscriptionType(Realm realm, SubscriptionType subType)
- public static async Task Init()
- public static Realm GetMainThreadRealm()
- public static Realm GetRealm()
- public static async Task RegisterAsync(string email, string password)
- public static async Task LoginAsync(string email, string password)
- public static async Task LogoutAsync()
- public static async Task SetSubscription(Realm realm, SubscriptionType subType)
- public static SubscriptionType GetCurrentSubscriptionType(Realm realm)

DialogService.cs

המחלקה DialogService היא מחלקת שירות סטטית לניהול הודעות (דיאלוגים) וחיווי משתמש באפליקציה .NET MAUI. למשל כאשר משתמש מנסה לעלות מסלול עם פחות מ-2 נקודות קופצת לו הדועה שהוא צריך להוסיף לפחות נקודה אחת נוספת. המחלקה כוללת את השיטות הבאות :

- `public static Task ShowAlertAsync(string title, string message, string accept)`
- `public static Action ShowActivityIndicator()`
- `public static Task ShowToast(string text)`

מחלקות של מודל תצוגה

במבנה של MVVM (Model-View-ViewModel), מחלקות שממוקמות בתיקיית ViewModels משמשות כמתווך בין הלוגיקה העסקית (Models, Services, Database) לבין הממשק הגרפי (UI). התקייה של ViewModels כוללת את המחלקות הבאות :

1. מחלקת- EditMapPinViewModel .
2. מחלקת- EditUserRecordViewModel .
3. מחלקת- MapsViewModel.cs .
4. מחלקת- UserRecordsViewModel.cs .
5. מחלקת- LoginViewModel.cs .

EditMapPinViewModel.cs .

ה ViewModel-הזה משמש לעריכת נקודות במפה, טעינת מסלול, והעלאת הנתונים לMongoDB Realm. וכוללת את השיטות הבאות :

- EditMapPinViewModel()
- void setMapName(string newMapName)
- void ApplyQueryAttributes(IDictionary<string, object> query)
- Task AddPointsToTrack()
- Task PrintList()
- static bool CheckMapNameAlreadyExists(string inputTrackName)
- Task UploadMapPinAndTrackObject()
- Task Cancel()

EditUserRecordViewModel.cs.

ה ViewModel-הזה משמש ליצירה של רשומה של זמני התוצאה של משתמש עבור מסלול שהוא ביצע במפה, טעינת רשומות של משתמשים לפ מסלול, והעלאת הנתונים לMongoDB Realm. וכוללת את השיטות הבאות :

- EditUserRecordViewModel()
- void ApplyQueryAttributes(IDictionary<string, object> query)
- Task SaveUserRecord(UserRecord newUserRecord)
- Task Cancel()

MapsViewModel.cs.

מחלקה זו שומרת בתוכה את השיטות שמאפשרות להראות את כל המסלולים ששמורים בענן של האפליקציה וכוללת בתוכה את השיטות הבאות :

- void deleteExistingMapPinFromCloude(Pin newPin, string mapNameToDelete)
- void OnAppearing()
- Task GoToUserRecordsList()
- Task Logout()
- static List<Pin> getPinsListByName(string trackName)
- Task ChooseMapFromList(MapPin map)
- Task CreateNewTrack()
- Task DeleteMap(MapPin pinOfChosenMap)
- Task DeleteUsersOfTrack(string trackName)
- Task DeleteSinglePin(MapPin pin)
- void Refresh()
- Task ToTimerPage()
- void ChangeConnectionStatus()
- Task<bool>WarningDeletingMyTrack(MapPin chosenPinOfMap)
- Task<bool>CheckMapOwnership(MapPin chosenPinOfMap)

UserRecordsViewModel.cs.

מחלקה זו נועדה בשביל להכין את כל הרשומות של המשתמשים שרשומים באפליקציה .

- `async Task DeleteUserRecord(UserRecord userRecordFromList)`
- `async Task ChooseRecordFromList(UserRecord userRecordFromList)`
- `async void OnAppearing()`
- `async Task Logout()`
- `async Task CreateNewTrack()`
- `async Task DeleteMap(MapPin pin)`
- `async Task DeleteSinglePin(MapPin pin)`
- `void Refresh()`
- `async Task ToTimerPage()`
- `void ChangeConnectionStatus()`
- `async Task GoToMapsList()`
- `void SortUserRecords()`
- `async Task<bool> CheckUserRecordOwnership(UserRecord userRecordFromList)`
- `IQueryable<UserRecord> getUserRecordsOfCurrentUser()`
- `IQueryable<UserRecord> getUserRecordsWithTheSameTrackName()`
- `async Task<bool> CheckItemOwnership(MapPin map)`

LoginViewModel.cs

מחלקה זאת אחראית על ההיתחברות הרשמה של המשתמש
לאפליקיה וכוללת את השיטות הבאות :

- public async Task Login()
- public async Task SignUp()
- private async Task DoLogin()
- private async Task DoSignup()
- private async Task<bool> VerifyEmailAndPassword()
- private async Task GoToMainPage()

שכבת ההצגה - קבצי xaml של Views

תיקיית Views בפרויקט NET MAUI מכילה את מסכי ממשק המשתמש של האפליקציה, לצורך ההיגיון. הוא ממלא תפקיד מכריע בבניית האפליקציה ובשמירה על ארכיטקטורת MVVM (Model-View-ViewModel) נקייה. תקייה זו כוללת את כל הדפים של הXAML.

שכבה זאת מאפשרת בעצם לבצע הפרדה בין השכבה הלוגית שכוללת את כל השיטות שנועדו בשביל להראות חישובים כגון המרחק שהמשתמש עבר או שמירת אובייקטים בענן. לשם כך שכבת ההצגה מיוצגת בתקייה Views.

עקרונות מנחים בשכבת ההצגה

- לכל דף תצוגה בקובץ ה-XAML יהיה, דף של קובץ ה-CS שבו ישמרו השיטות שבהם דף התצוגה משתמש .
- כל הכפתורים והעברה לדפים אחרים יבוצעו על ידי שיטות שעובדת עם RelayCommand . RelayCommand [RelayCommand] הוא תכונה (Attribute) בספריית CommunityToolkit.Mvvm, שנועד להגדיר פקודות (Commands) באופן פשוט ונוח בתוך ה- ViewModel ב NetMaui. פקודות (ICommand) משמשות להפרדה בין הלוגיקה של האפליקציה לבין קוד ה UI- ומאפשרות חיבור בין כפתורים לאירועים מבלי לכתוב קוד ישירות בקובץ ה- xaml.cs.
- השימוש ב- RelayCommand חוסך כתיבה מיותרת על ידי הפיכת מתודות רגילות לפקודות אוטומטית, במקום ליצור ICommand באופן ידני.
- השימוש במטודה מסוג RelayCommand בקובץ ה-xaml, תכלול הוספה של המילה השמורה Command לשם המטודה .

דפים המשמשים להצגה

הקבצים שמופיעים כחלק משכבת ההצגה יהיו בתקייה של Views, כל המחלקות בתקייה הזאת יורשות מהמחלקה האבסטרקטית ContentPage. כוללת את התיקיות הבאות והקבצים הבאים :

- תקיית MapSettings-הכוללת בתוכה את הדפים שאחראים להגדרות של המסלולים , והגדרות הכוללות הלעאת מסלול לשרת.

1. דף - AddMapToDbPage.xaml .
2. דף - DistancePage.xaml .
3. דף - EditPinAddr.xaml .
4. דף - MapPage.xaml .
5. דף - MapsPage.xaml .

- תקיית UserRecords-תקייה זו כוללת את הדפים שקשורים לאובייקט מסוג UserRecord.תקייה זאת כוללת את הקבצים הבאים :

1. דף-AddRecordToDb.xaml
2. דף-TimerPage.xaml
3. דף-UserDetails.xaml
4. דף-UserRecordsPage.xaml

AddMapToDbPage.xaml.cs

מחלקה זו מייצגת את הדף שבו אנו קובעים את שם המסלול ולאחר לחיצה על כפתור ה-OK, מעלים אותו לשרת.

.DistancePage.xaml.cs

מחלקה המייצגת את המרחק של המסלול שהמשתמש בנה או בחר מהרשימה הקיימת. הדף יציג את המרחק בין נקודה לנקודה בק"מ ואת המרחק הכללי של המסלול.

EditPinAddr.xaml.cs

מאחר שלכל PIN על גבי המפה יש כתובת, בעזרת דף זה ניתן לשנות את הכתובת ושם הפין - כאשר ברירת המחדל היא שהכתובת של הפינים היא לפי מספר כאשר הפין הראשון מסומן כמספר 1 וכך הלאה. המחלקה כוללת את השיטות הבאות:

`setPinsList(List< Pin> newPinsList)`

`PinExists(string pinLabel)`

`setAddress(string pinLabel, string newAddress)`

`OnDoneButtonClicked(object sender, EventArgs e)`

MapPage.xaml.cs

מחלקה זו מייצגת את המפה של Google.maps ומציגה בתוכה את כל הכפתורים שהצגנו בחלק הקודם (הכפתורים הם הלעאת המסלול לשרת, חישוב המרחק של המסלול וכן הלאה). השיטות של המחלקה הן:

- public static MapPage Instance { get; }
- public string MapTitle { get; set; }
- public void SetTitle(string newTitle);
- public void setPinsList;()
- public async Task<bool> IsLocationEnabled;()
- public List< Pin> GetPinList;()
- private List< Pin> GetPins;()
- private void numberOfPoints(object sender, MapClickedEventArgs e);
- public void CancelRequest;()
- public async Task GoToTimerPageButton_Pressed;()
- public async Task ZoomToMyLocationButton_Pressed;()
- public async Task GoToUserRecordsListButton_Pressed;()
- public async Task DeletLastPointButton_Pressed;()
- public async Task AddToCloudButton_Pressed;()
- public async Task CalcDistanceButton_Pressed;()
- public async Task ResetMapButton_Pressed;()
- public async Task EditPointButton_Pressed;()
- private void addPointOnMap(object sender, MapClickedEventArgs e);
- private void PrintPinAddresses(object sender, MapClickedEventArgs e);
- public void ShowButtonsOnMap(bool cond);
- public void ShowStartExerciseButton(bool cond);
- public void ShowUsersRecordsButton(bool cond);
- public void ShowTrack_Clicked;()
- private async Task<bool> EnoughPins(int num);
- public void ClearMap;()
- private async void toPage(string pageName);
- private double GetDistance(Maui.GoogleMaps.Pin p1, Maui.GoogleMaps.Pin p2);

MapsPage.xaml.cs

מחלקה זו מייצגת את הדף שבו מוצגים כל המסלולים הקיימים
בשרת . ואפשרות למחיקת מסלולים קיימים של המשתמש שמחובר
לאפליקציה

AddRecordToDb.xaml.cs

מחלקה זו כוללת את הפרטים שקשורים לרשומה של הפעילות
האירובית שהמשתמש ביצע וכוללת את השיטות הבאות :

setRecordUserTime(string newRecordUserTime)

setTrackName(string newTrackName)

UploadUserRecordObject()

TimerPage.xaml.cs

מחלקה זו מתארת את הטיימר שנועד בשביל לחשב את משך זמן
הפעילות וכולל את הכפתורים הבאים : הפעלת הטיימר, השהיית
הטיימר, איפוס הטיימר, והעלאת הרשומה לשרת . המחלקה כוללת את
השיטות הבאות :

setTitle(string newTitle)

OnTimerElapsed()

StartTimer() ()

PauseTimer()

ResetTimer()

Upload()

דפוסי עיצוב (Design Patterns)

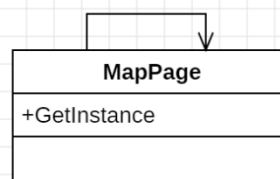
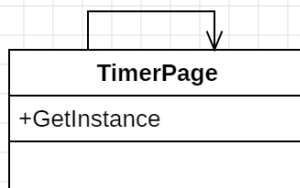
SingletonPattern

תבנית עיצוב Singleton (סינגלטון) היא דפוס תכנון Design (Pattern) בתחום התכנות שמטרתו להבטיח שלמחלקה מסוימת תהיה אובייקט יחיד בלבד בכל התוכנית, ולספק גישה גלובלית אליו.

בתבנית זו נעשה שימוש במחלקות הבאות :

1) MapPage-מחלקה זו מייצגת את המפה של GOOGLE MAPS. מכיוון שבכל רגע נתון המשתמש יכול לקבל גישה למפה אחת של GOOGLE MAPS, בין אם זה בכדי לבנות מסלול או להעיזר במפה בשביל לבחור מסלול בכדי לבצע פעילות אירובית כלשהי .

2) TimerPage-מחלקה זו מייצגת את הטיימר של האפליקציה . מכיוון שבשביל למנוע מצב שבו משתמש בוחר מסלול מסויים ואז מחליט לבחור מסלול חדש אז שלא ייווצר מצב שבו נעזר בזמן של המסלול הקודם יש צורך שבכל פעם שהמשתמש רוצה להפעיל את הטיימר חייב שיהיה אך ורק מופע אחד של הטיימר .



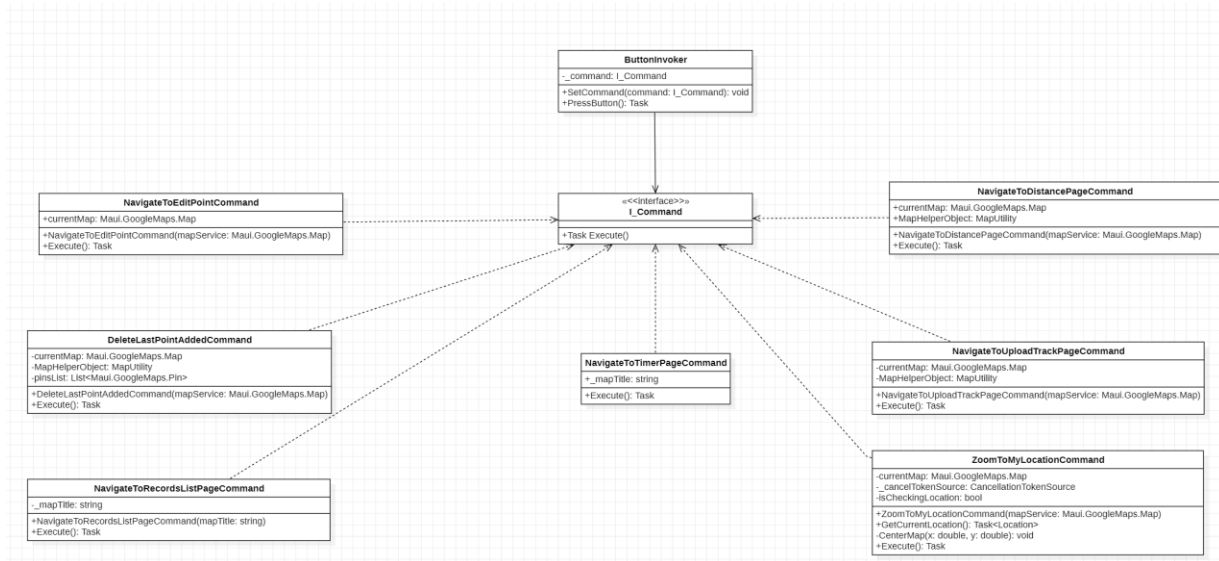
Command pattern

הדפוס Command (או "דפוס פקודה") הוא דפוס עיצוב המתמקד בהפרדה בין אובייקט המנחה לבצע פעולה לבין הקוד שמבצע את הפעולה עצמה.

במקום שהקוד המבצע את הפעולה יהיה ישירות בתוך האובייקט שמנחה אותה (למשל, כשלוחצים על כפתור והפעולה מתבצעת מיד), הדפוס מפריד את שני המרכיבים האלה. במקום זאת, כל פעולה (או פקודה) מנותבת לאובייקט נפרד שנקרא Command (פקודה),

MapPage-מחלקה זו כוללת בעצם כוללת בעצם את כל הכפתורים שמופיעים על גבי המפה של הגוגל מפיס. לשם כך עבור כפתור בנינו מחלקה נפרדת שמטפלת בפקודה עצמה. למשל עבור הכפתור שמחשב את המרחק של המסלול בנינו את המחלקה NavigateToDistancePageCommand שבעצם מייצגת את הכפתור שמעביר את המשתמש לדף שבו מוצג המרחק של המסלול שאותו הוא בנה על גבי המפה של Google.Maps. בתבנית זו נעשה שימוש בממשק ובמחלקות הבאות:

1. ButtonInvoker - מחלקה המאפשרת להפעיל את הפקודות שמופיעות על גבי המפה בצורה גנרית וגמישה.
2. DeleteLastPointAddedCommand - מחלקה שמורידה את הנקודה האחרונה שהוספה על גבי המסלול.
3. NavigateToDistancePageCommand - מחלקה שמעבירה את המשתמש לדף שמציגה את המרחק של המסלול ואת המרחקים בין נקודה לנקודה של המסלול שהמשתמש בנה.
4. NavigateToEditPointCommand - מחלקה שמעבירה את המשתמש לדף שמאפשר לערוך את השם של הנקודה.
5. NavigateToRecordsListPageCommand - מחלקה שמעבירה את המשתמש לדף שמציג את כל הרשומות של המשתמשים.
6. NavigateToTimerPageCommand - מחלקה שמעבירה את המשתמש לדף שבו מוצג הטיימר של האפליקציה.
7. NavigateToUploadTrackPageCommand - מחלקה שמעבירה את המשתמש לדף שמאפשר לעלות את המסלול לשרת.
8. ZoomToMyLocationCommand - מחלקה שמאפשרת למשתמש לעשות ZOOM למיקום של המשתמש.
9. I_Command - ממשק שממש את הפקודה של Excute.



Strategy Pattern

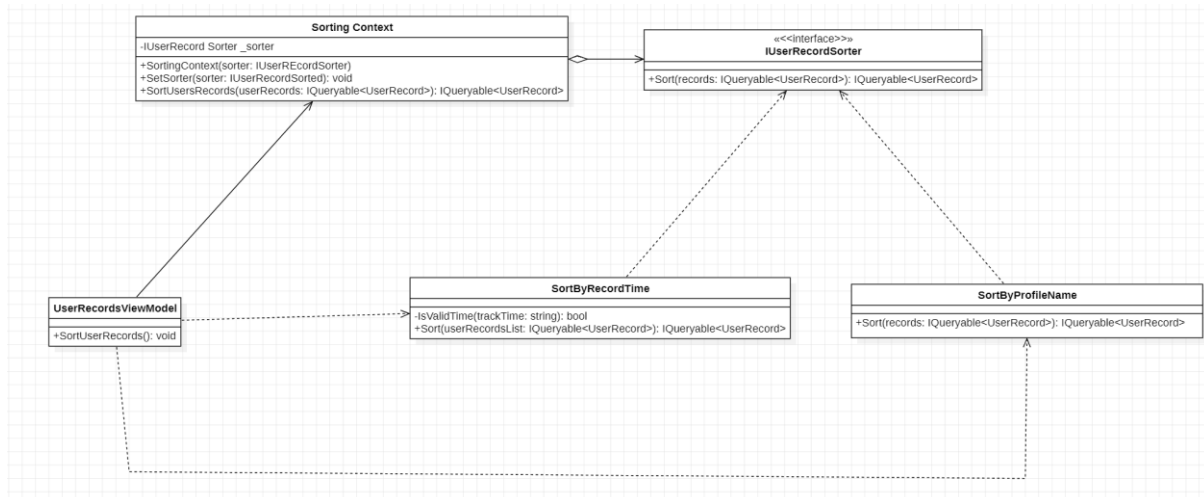
תבנית עיצוב "אסטרטגיה" (Strategy Design Pattern) היא תבנית עיצוב מבני שמטרתה להפריד בין אלגוריתם (אסטרטגיה) לבין הקוד שמשמש בו. במקום לקבוע אלגוריתם קבוע בתוך קוד מסוים, תבנית העיצוב מאפשרת לבחור ולהחליף אסטרטגיות שונות במהלך הריצה, מבלי לשנות את הקוד שבונה את המערכת.

במילים פשוטות, תבנית העיצוב הזו מאפשרת לבחור את הדרך שבה תתבצע פעולה כלשהי בזמן הריצה, ובכך מקלה על הוספת אסטרטגיות חדשות או שינוי אסטרטגיות קיימות מבלי לשנות את הקוד שהן פועלות בו.

דוגמה נפוצה לשימוש בתבנית זו היא במערכות שבהן יש אלגוריתמים שונים לפתרון בעיות שונות, כמו חישוב מחירים, חיפוש או מיון, ותצטרך לבחור באלגוריתם המתאים ביותר בהקשר נתון.

UserRecordsViewModel - בגלל שהאפליקציה כוללת בתוכה את האפשרות למיון של רשומות המשתמשים. במחלקה זו ישנה את השיטה SortUserRecords - שמאפשרת למיין (לפי SelectedSortOption) את הרשימה UserRecordsList (שמציגה את הרשומות של המשתמשים באפליקציה) לפי זמן הפעילות, שם המשתמש שביצע את הפעילות או זמן העלאת הרשומה לענן של מונגו די בי. בתבנית זו נעשה במחלקות ובמשק הבאים:

1. IUserRecordSorter - ממשק שכולל בתוכו את השיטה Sort שמקבלת רשימה של אובייקטים מטיפוס UserRecord ומחזירה רשימה ממויינת לפי המחלקה שבה היא שומשה.
2. Sorting Context - המחלקה SortingContext משמשת כקונטקסט למיון (Sorting) של רשומות משתמשים UserRecord.
3. SortByProfileName - מחלקה שממיינת את רשימת UserRecord לפי שם המתשמש.
4. SortByRecordTime - מחלקה שמיינת את רשימת UserRecord לפי משך זמן הפעילות.



Observer Pattern

עיצוב "צופה" (Observer Design Pattern) היא תבנית עיצוב התמקדת בקשר בין אובייקטים, כך שכאשר מצב של אובייקט משתנה, כל הצופים (Observers) שמנויים על אותו אובייקט מעודכנים אוטומטית על השינוי.

במילים פשוטות, תבנית העיצוב הזו מאפשרת לאובייקטים לעקוב אחרי שינויים באובייקט אחר בלי שהם יצטרכו להיות תלויים אחד בשני באופן ישיר. ברגע שהאובייקט הנתון משתנה, כל הצופים שמנויים עליו מקבלים את העדכון ומבצעים פעולה בהתאם.

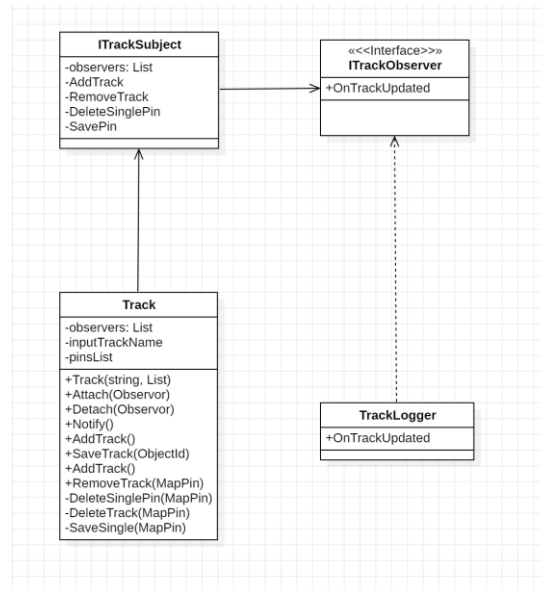
דוגמה נפוצה לשימוש בתבנית זו היא במערכות המידע, שבהן יש צורך לעדכן מספר רכיבים (כמו ממשקי משתמש, מערכות ניהול או שירותים אחרים) כאשר מתרחשים שינויים במידע המרכזי. כחלק מהסיפרייה של CommunityToolkit.MVVM ישנו מאפיין שנקרא ObservableProperty. ObservableProperty הוא רכיב מרכזי ביישום תבנית העיצוב "צופה", שבה שינויים בתכונות מעדכנים אוטומטית את הממשק המשתמש (UI).

Track-זאת מחלקה שבעצם בעזרתה ניתן לעלות אובייקטים מסוג TrackMongo1-שזהו אובייקט שנעלה לענן של MONGO DB ששומר את פרטי המסלול, וניתן לעלות את הרשימה של MAPPIN שיוצרים את המסלול עצמו.

TrackLogger-מחלקה שמשמשת למעקב אחרי שינויים של המחלקה Track.

ITrackSubject-הממשק ITrackSubject הוא חלק מתבנית העיצוב (Observer) המשקיף ומשמש כנושא (Subject) שמודיע ל"משקיפים" (Observers) על שינויים.

ITrackObserver-הינו ממשק שהוא חלק מתבנית העיצוב של (Observer) המגיב לשינויים בנושאים (Subjects) מסוימים. במקרה הזה מגיב לשינויים של האובייקט Track.



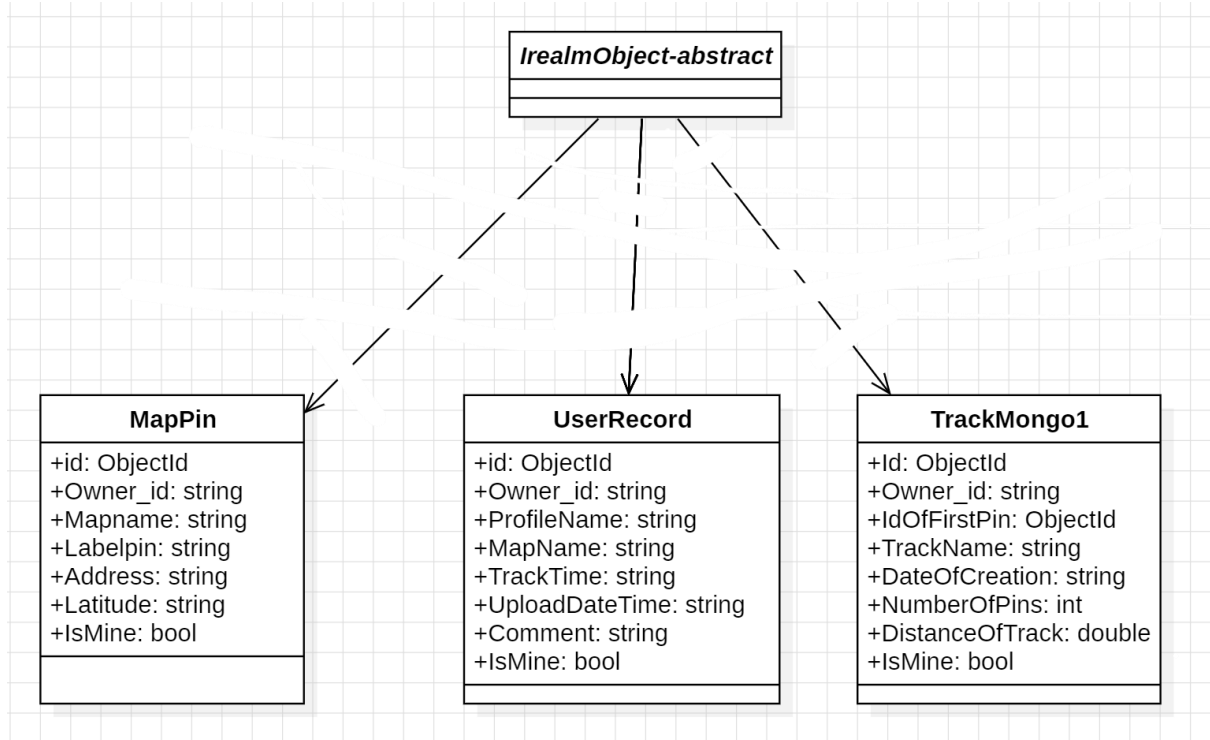
שינויים עתידיים אפשריים

השימוש ב Design Patterns שהצגנו מקודם יעזור לנו להוסיף שינויים באפליקציה .

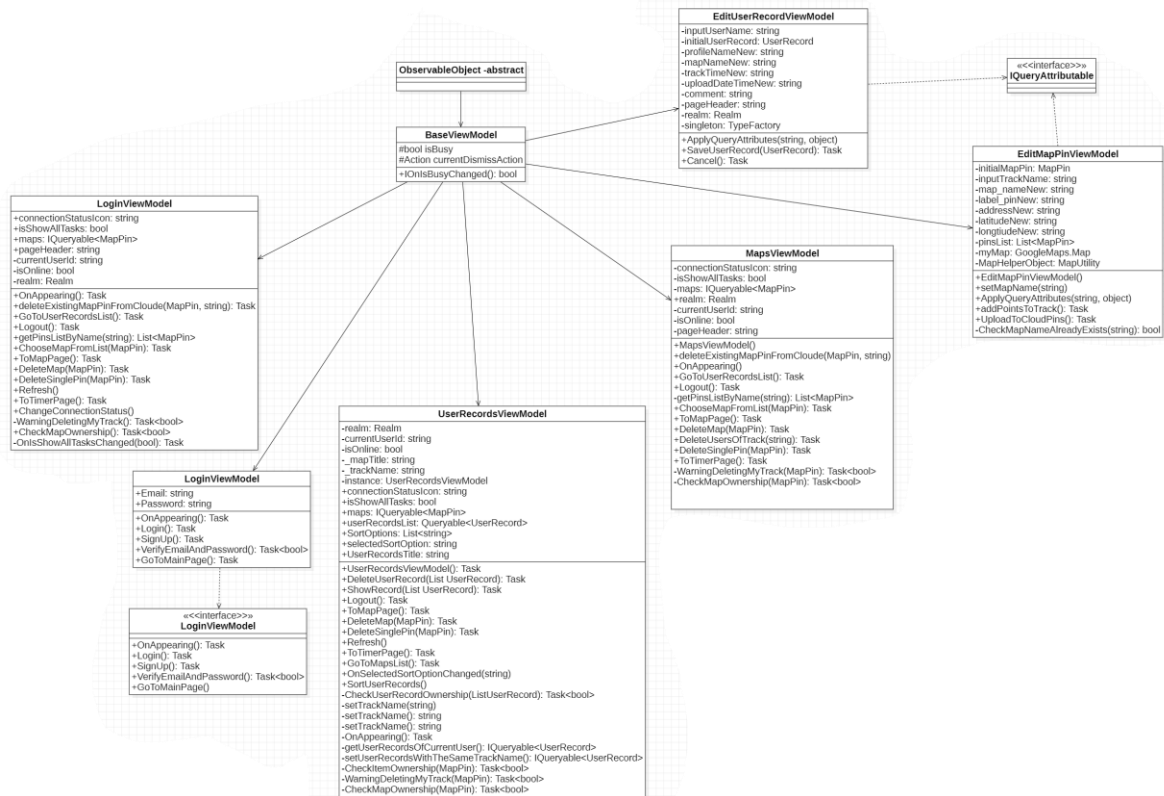
1. מימוש מחלקת Track בעזרת דפוס העיצוב של Observer מאפשרת גמישות בסוג של האובייקטים שאנו רוצים לעלות לאפליקציה , למשל אם נירצה לשנות את סוג המסלול כך שהוא יהיה זמין רק עבור קבוצה ספציפית של משתמשים פשוט נוכל להוסיף מצב חדש שיקרא AddToGroup-אשר יבחר את המסלולים המתאימים ואת הנקודות המתאימים עבור המתשמשים של אותה הקבוצה .
2. עבור רשימת הרשמות של זמני הפעילות של המשתמשים שבאפליקציה שמימשנו בעזרת Strategy Design Pattern במחלקה UserRecordsList, אם בהמשך נירצה להוסיף לאפליקציה גם את סוג הפעילות האירובית למשל רכיבה על אופניים אז בשביל להראות את הרשומות של המשתמשים שביצעו פעילות אירובית מסוג רכיבה על אופניים, כל שנותר לנו הוא להוסיף מחלקה בשם SortByActivity שתמין את הרשומות לפי הפעילות האירובית.
3. על גבי דף המפה של Google.Maps מימשנו כפתורים שונים בעזרת ה Command Design Pattern, מימוש זה מקל עלינו כי במידה ונרצה להוסיף עוד כפתור על גבי המפה למשל כפתור שיראה אילו משתמשים נמצאים כרגע באפליקציה אז כל שנותר לנו הוא רק להוסיף מחלקה בשם :
NavigateToOnlineUseresCommand שתעביר את המשתמש לדף שבו יוצגו כל המשתמשים שנמצאים כרגע באפליקציה .
4. על ידי שימוש העלאת האובייקטים לMONGODB ניתן בקלות לשנות את האפליקציה כך שתוכל לעלות אובייקטים שונים . למשל אם נירצה לעלות אובייקט עבור מסלולים של תחריות או ניווט .

דיאגרמות

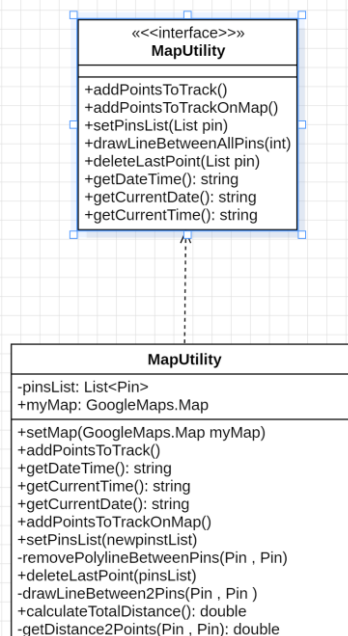
דיאגרמות של המחלקות שמעלים לענן של mongo db atlas.



דיאגרמות של ViewModels



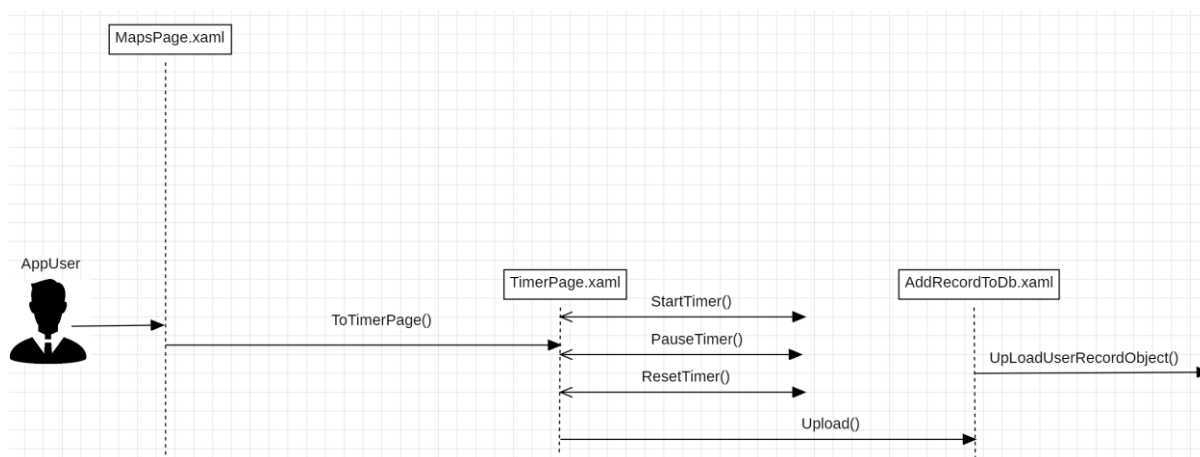
מחלקת עזר לחישובים



דיאגרמת של דפי Views

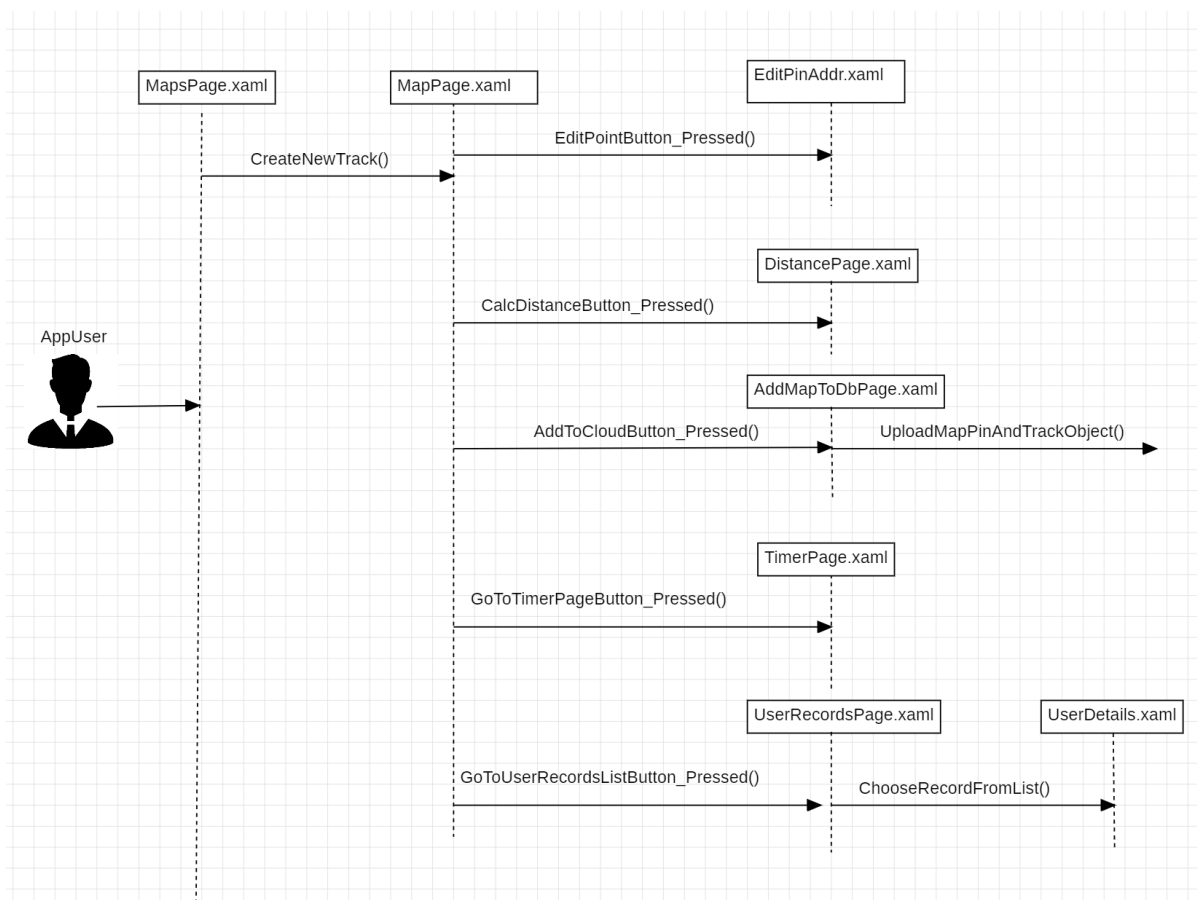
בדיאגרמות הבאות נראה את הדפים של Views, שאלו בעצם דפי XAML שהמשתמש רואה שמקשרות בין ממשק המשתמש לפונקציות שמעבירות את המשתמש לדפי XAML. הנחוצים.

דיאגרמת רצף 1- התחלת טיימר



דיאגרמה זו מציגה את האפשרות כאשר המשתמש לוחץ על כפתור ה**Timer** בדף הראשי.

דיאגרמת רצף 2 - בחירת מסלול קיים



דיאגרמת רצף 3 - יצירת מסלול חדש

